

task-1 : 环境配置

首先,请你下载好 python解释器 或 conda ,以下教程将基于 conda

1. 参考以下网址, 完成 conda 环境的下载与配置

1. [Anaconda安装与配置全攻略 \(2025版\) - 知乎](#): 学习conda的下载流程
2. [CUDA学习之路1\]——速通环境配置 - 杜子源的博士居](#): 学习Pytorch和Triton配置
3. [pip修改国内镜像源 \(临时/永久\) - 知乎](#): python换源教程

2. 创建一个python环境

3. 在终端 (cmd,powershell) 中输入

```
python -c "print('Hello Python')" #预期输出Hello Python
```

4. 选择一个你喜欢的IDE,如 vscode , pycharm , CLion ,在其上配置好python解释器

5. 学习基本的python语法

推荐网站: [Python 基础教程](#) | [菜鸟教程](#)

6. (可选) 完成AI工具配置

我们建议, 在本地电脑上下载 claude 或者 codex 的命令行工具,下载流程如下

1. 前往 Node.js 官网下载好 Node.js [Node.js — 下载 Node.js®](#), 并添加到环境变量中
2. 下载 claude 或者 codex , 使用如下命令下载

```
#claude的下载
npm install -g @anthropic-ai/claude-code

#codex的下载
npm install -g @openai/codex
```

上述下载过程请参考:

[Claude CLI 从安装到使用完整教程 \(Windows 版\) Claude Code 是 Anthropic 官方推出 - 掘金](#)

[2026 最新 Codex CLI 国内安装与使用全攻略 \(Windows / Mac / Linux\) - 知乎](#)

3. 如果没有 claude 和 codex 的 api 密钥 该怎么办 🤖

前往[CC Switch 官方网站 - AI 编程工具统一管理平台](#),下载CC Switch, 可能需要科学上网, 然后你就可以用 Claude CIL 和 Codex CIL 接入其他模型

CC Switch的配置请参考: [CC Switch下载、安装和配置教程 \(图解, 超级详细\) - 知乎](#)

注: 本次题目不对编程语言有要求, 你也可以使用C++等你喜欢编程语言完成以下题目

task-2 : opencv与pillow的使用

背景:

在计算机视觉（CV）领域，数据的预处理与图像的基本操作是构建所有上层算法的基石。无论是传统的图像处理，还是基于深度学习的目标检测与分割，第一步永远是“如何正确地读入一张图”以及“如何对其进行最基本的变换”。本题目将带领你逐步了解 opencv 与 pillow 这两个最基础的图像库的使用,学会图像处理的基本技能。

题目:

1. 使用opencv+pillow打开并显示任意一张图片，并分离输出图片的三个通道
2. 对图片进行**缩放，旋转，平移，翻转**操作
3. 使用opencv,检测**example1.jpg**的轮廓并保存，同时计算轮廓的总面积和总周长
4. 使用opencv进行对**example2.mp4**运动目标检测,比较使用**MOG2 背景减除**与**未使用背景减除**对运动物体识别的影响
5. 使用opencv的tracker(追踪器)，对**example3.mp4,example4.mp4**中小球进行追踪，保存追踪结果视频，并思考**如何提高追踪准确度**

提交材料:

1. 项目源码
2. **example2.mp4**与**example3.mp4**与**example4.mp4**的检测结果视频
3. 一份文档，其中包含
 - 题一，题二图像操作结果
 - 题三绘制有轮廓的图像与总面积和总周长数据



- 题四的两种方法处理的**标注视频与分析**
 - 题五的**标注视频**与结果分析
-

task-3 : MLP识别MNIST数据集

背景：

MNIST 被誉为计算机视觉界的“Hello World”。本次任务要求大家利用 **多层感知机（MLP）** 来完成对该数据集的分类识别。虽然 MLP 并非最强的视觉模型，但它包含了神经网络最核心的前向传播、反向传播与梯度更新机制。通过实现一个简单的 MLP，你将深刻理解“数据如何流入网络”、“损失如何回传”以及“模型如何拟合”这三个贯穿整个深度学习生涯的基本问题。

题目：

1. 理解深度学习的各种概念

神经网络基础

- 神经元 / 感知机
- 权重 & 偏置
- 激活函数
- 前向传播
- 隐藏层 & 输出层
- 全连接层

训练与优化

- 损失函数
- 梯度下降
- 混合精度训练
- 反向传播（链式法则）
- 学习率
- 优化器
- Epoch、Batch、Iteration

关键技巧与调参

- 过拟合 & 欠拟合
- 正则化（L1、L2、Dropout）
- 批归一化（Batch Normalization）
- 训练集 / 验证集 / 测试集

基础工具与概念

- Tensor / 张量
- 自动求导（Autograd）
- GPU 与CPU区别

2. 使用pytorch的自动求导与梯度下降,完成最小二乘法拟合

使用梯度下降，完成对下述二元函数的拟合：

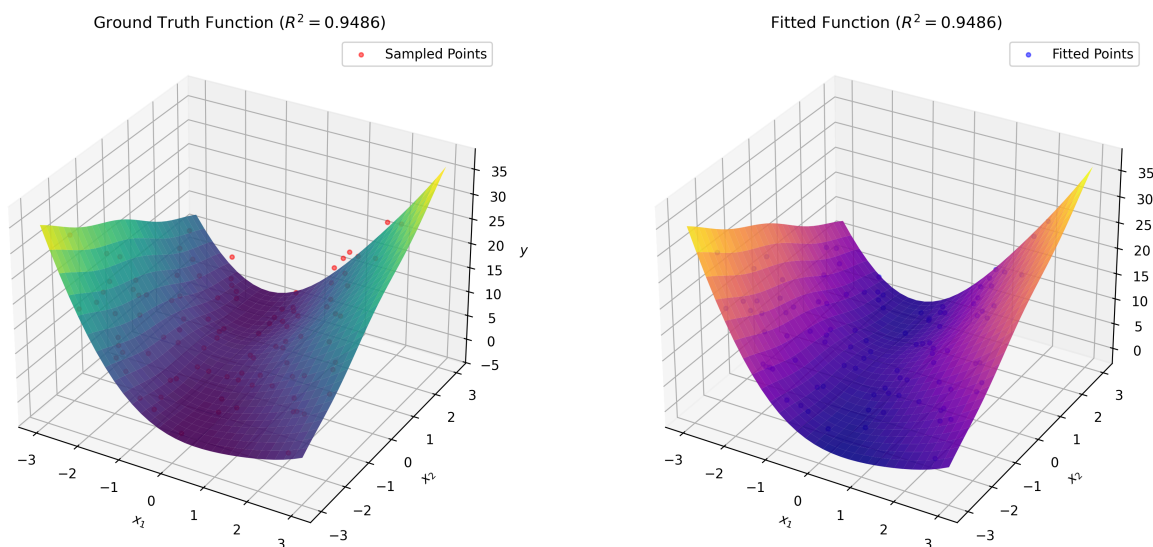
$$y = 2x_1^2 + 1.5x_1x_2 + 3\sin(x_1) + 0.5\cos(2x_2) + e^{-0.5x_1} + 0.8x_2 + 2026 + \text{噪声(在 } -3 \text{ 到 } 3 \text{ 的随机数)}$$

要求绘制出原函数(不包含噪声)真实图像, 与pytorch拟合的函数图像,同时输出拟合获得的参数值与决定系数: R^2

参数如下:

$$y = a_1 x_1^2 + a_2 x_1 x_2 + a_3 \sin(x_1) + a_4 \cos(2x_2) + e^{a_5 x_1} + a_6 x_2 + b$$

绘图示例:



3. 搭建简单的多层感知机(MLP)

使用 pytorch 与 pandas 和图像库, 构建简单的多层感知机, 读取数据, 完成手写字体的识别

要求绘制模型训练的损失函数(Loss),训练准确度(Train Acc),测试准确度(Test Acc), 与可视化至少3张图片结果

提交材料:

1. 项目源码
2. 一份文档, 其中包含
 - 题一: 你对深度学习的各种概念的理解
 - 题二: 原函数(不包含噪声)真实图像, 与pytorch拟合的函数图像,拟合获得的参数值与决定系数: R^2
 - 题三: 模型训练的损失函数(Loss),训练准确度(Train Acc),测试准确度(Test Acc)图片, 与可视化结果

task-4 : 自主选题

大一同学从4个方向中任选1个完成, 大二同学任选2个完成,完成提高部分的我们将优先考虑!!!

1. (目标追踪与识别) 基于YOLO的目标识别与模型量化

背景：随着深度学习的快速发展，目标检测技术在各个领域得到了广泛应用，如自动驾驶、智能监控、医疗影像分析等。YOLO (You Only Look Once) 作为一种经典的目标检测算法，以其**速度快、精度高**的优势，成为工业界和学术界最受欢迎的目标检测框架之一。相信你也从task2的example5.mp4中,发现仅用opencv识别被遮挡和具有复杂轨迹的物体效果很差,这时使用YOLO来进行目标追踪与识别是一种很好的解决方法。本题将基于YOLO模型，实现对无人机目标的追踪。

题目：

1. 前往[Ultralytics官网](#),学习YOLO的基本 api 接口，学会如何训练与使用YOLO模型进行推理
2. 使用YOLO官方的预训练模型权重对 目标追踪识别方向 example文件夹下的三张图片进行推理保存
3. 基于我们所提供的**无人机数据集与无人机预训练权重 drone.pt** 进行训练,导出并保存模型与训练结果绘图
4. 对 example目录 无人机飞行视频进行推理，分析 ([EAO](#) (可选)，**模型大小(MB)**，**推理速度(fps)**) 指标
5. (**提高**) 对导出模型进行 fp16 或 int8 量化及 onnx 或 TensorRT 导出，在保证精确度下，提高模型推理速度

提交材料：

1. 项目源码
2. 推理好的图片与视频
3. 一份文档，其中包含
 - 模型训练的**损失函数(Loss)**,**训练准确度(Train Acc)**,**测试准确度(Test Acc)**，与混沌矩阵图片
 - 指标数据及分析
 - (**如有完成提高部分**)导出的**模型大小与推理速度**与原模型比较分析

推荐数据集: drone and bird <https://app.roboflow.com/ds/nSMQrzUp0X?key=TTcKC8vGXl>

2. 基于卷积和 Transformer 的图像分类

背景:图像分类是计算机视觉中的基础任务之一，广泛应用于工业检测、遥感识别、医学影像分析、智能安防等场景。传统卷积神经网络能够有效提取局部纹理与边缘特征，而 Transformer 结构具有较强的全局建模能力。结合卷积网络与 Transformer 方法，有助于提升模型对复杂图像场景的分类准确率与泛化能力，具有重要的研究价值和应用前景。

题目：

1. 请基于深度学习方法，设计并实现一种卷积神经网络图像分类模型，对图像数据集进行分类训练与测试.
2. (**提高**) 对原模型结构进行改进，例如引入注意力机制、数据增强或轻量化模块，在保证模型准确率的同时，提高模型泛化能力或推理速度。**了解Vit原理,无需使用这个架构**

提交材料：

1. 项目源码
2. 分类结果可视化图片
3. 一份文档，其中包含
 - 模型结构设计说明

- 模型训练的 **损失函数(Loss)**、**训练准确率(Train Acc)**、**测试准确率(Test Acc)** 曲线
- 模型分类结果分析
- **(如有完成提高部分)** 改进方法及实验对比分析

推荐数据集： Intel Image Classification <https://www.kaggle.com/datasets/puneet6060/intel-image-classification>

3. 时序模型

背景： 时序数据广泛存在于工业监测、气象预测、金融分析、交通流量预测、设备故障诊断等实际应用场景中。时序模型的核心目标是从历史数据中学习时间依赖关系，并对未来状态或类别进行预测。循环神经网络、LSTM、GRU、Temporal CNN 以及 Transformer 等模型均可用于时序建模。通过构建有效的时序预测或分类模型，可以提升系统对动态变化过程的分析与预测能力。

题目：

1. 请基于深度学习方法，设计并实现一种时序模型，可选择 RNN、LSTM、GRU、TCN 或 Transformer 等结构，完成一个时序预测或时序分类任务。要求给出模型训练过程中的 **Loss 曲线**，并使用合适的评价指标对模型效果进行分析，例如 **MAE**、**MSE**、**RMSE**、**Accuracy** 等。
2. **(提高)** 对模型进行优化，例如引入注意力机制、多层时序特征提取、滑动窗口预测、多变量输入、归一化处理或模型轻量化方法，在**保证预测精度**的同时，提高模型的**稳定性与推理速度**。

提交材料：

1. 项目源码
2. 时序预测结果或分类结果可视化图片
3. 一份文档，其中包含
 - 数据集介绍与预处理方法
 - 模型结构设计说明
 - 模型训练的 **损失函数(Loss)** 曲线
 - 模型评价指标结果，如 **MAE**、**MSE**、**RMSE** 或 **Accuracy**
 - 预测曲线与真实曲线对比图，或分类结果混淆矩阵
 - **(如有完成提高部分)** 模型改进方法及实验分析

推荐数据集： Air Passengers Dataset <https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv>

4. 图像生成(去雾算法实现)

背景： 恶劣天气条件下的图像退化是计算机视觉领域长期面临的挑战之一。在智能交通、遥感监测、户外监控等实际应用场景中，雾霾天气会导致采集到的图像出现对比度下降、色彩偏移、细节模糊等问题，严重影响后续目标检测、语义分割等高层视觉任务的性能。图像去雾作为底层视觉增强的关键预处理环节，旨在从单张或多张有雾图像中恢复出清晰的场景内容，具有重要的研究价值与应用前景。常见有GAN,CNN,Transformer,Diffusion等模型

题目：

1. 请你基于图像处理或深度学习技术，设计并实现一种轻量级去雾算法，在保证去雾效果的同时兼顾处理速度，还原清晰真实场景。要求测量模型生成图片的**PNSR**及模型的**推理速度**

2. (提高) 对导出模型进行 fp16 或 int8 量化, 在保证精确度下, 提高模型推理速度
3. 模型推荐结构,你也可以在此基础上改进或采用新的模型结构:

```
# ===== 3. 生成器 (U-Net) =====
class UNetDown(nn.Module):
    def __init__(self, in_channels, out_channels, normalize=True, dropout=0.0):
        super().__init__()
        layers = [
            nn.Conv2d(in_channels, out_channels, kernel_size=4, stride=2,
padding=1),
            nn.LeakyReLU(0.2, inplace=True)
        ]
        if normalize:
            layers.append(nn.BatchNorm2d(out_channels))
        if dropout:
            layers.append(nn.Dropout2d(dropout))
        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)

class UNetUp(nn.Module):
    def __init__(self, in_channels, out_channels, dropout=0.0):
        super().__init__()
        layers = [
            nn.ConvTranspose2d(in_channels, out_channels, kernel_size=4,
stride=2, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        ]
        if dropout:
            layers.append(nn.Dropout2d(dropout))
        self.model = nn.Sequential(*layers)

    def forward(self, x, skip_input):
        x = self.model(x)
        x = torch.cat((x, skip_input), 1)
        return x

class DehazeGenerator(nn.Module):
    def __init__(self, in_channels=3, out_channels=3):
        super().__init__()
        # 下采样
        self.down1 = UNetDown(in_channels, 64, normalize=False)
        self.down2 = UNetDown(64, 128)
        self.down3 = UNetDown(128, 256)
        self.down4 = UNetDown(256, 512, dropout=0.5)
        self.down5 = UNetDown(512, 512, dropout=0.5)
        self.down6 = UNetDown(512, 512, dropout=0.5)
        self.down7 = UNetDown(512, 512, dropout=0.5)

        self.bottleneck = nn.Sequential(
            nn.Conv2d(512, 512, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(inplace=True),
            nn.Dropout2d(0.5)
```

```

    )

    # 上采样
    self.up1 = UNetUp(512, 512, dropout=0.5)
    self.up2 = UNetUp(1024, 512, dropout=0.5)
    self.up3 = UNetUp(1024, 512, dropout=0.5)
    self.up4 = UNetUp(1024, 512)
    self.up5 = UNetUp(1024, 256)
    self.up6 = UNetUp(512, 128)
    self.up7 = UNetUp(256, 64)

    self.final = nn.Sequential(
        nn.ConvTranspose2d(128, out_channels, kernel_size=4, stride=2,
padding=1),
        nn.Tanh()
    )

def forward(self, x):
    d1 = self.down1(x)
    d2 = self.down2(d1)
    d3 = self.down3(d2)
    d4 = self.down4(d3)
    d5 = self.down5(d4)
    d6 = self.down6(d5)
    d7 = self.down7(d6)
    bottleneck = self.bottleneck(d7)

    u1 = self.up1(bottleneck, d7)
    u2 = self.up2(u1, d6)
    u3 = self.up3(u2, d5)
    u4 = self.up4(u3, d4)
    u5 = self.up5(u4, d3)
    u6 = self.up6(u5, d2)
    u7 = self.up7(u6, d1)
    out = self.final(u7)
    return out

# ===== 4. 判别器 (PatchGAN) =====
class Discriminator(nn.Module):
    def __init__(self, in_channels=3):
        super().__init__()
        # 输入: [有雾图, 生成图/清晰图] → 6通道
        # 注意: 移除了最后的Sigmoid层, 配合BCEWithLogitsLoss使用
        self.model = nn.Sequential(
            # 第一层: 不归一化
            nn.Conv2d(in_channels * 2, 64, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),

            # 第二层
            nn.Conv2d(64, 128, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, inplace=True),

            # 第三层
            nn.Conv2d(128, 256, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(256),

```

```

nn.LeakyReLU(0.2, inplace=True),

# 第四层
nn.Conv2d(256, 512, kernel_size=4, stride=1, padding=1),
nn.BatchNorm2d(512),
nn.LeakyReLU(0.2, inplace=True),

# 输出层 - 移除Sigmoid
nn.Conv2d(512, 1, kernel_size=4, stride=1, padding=1),
)

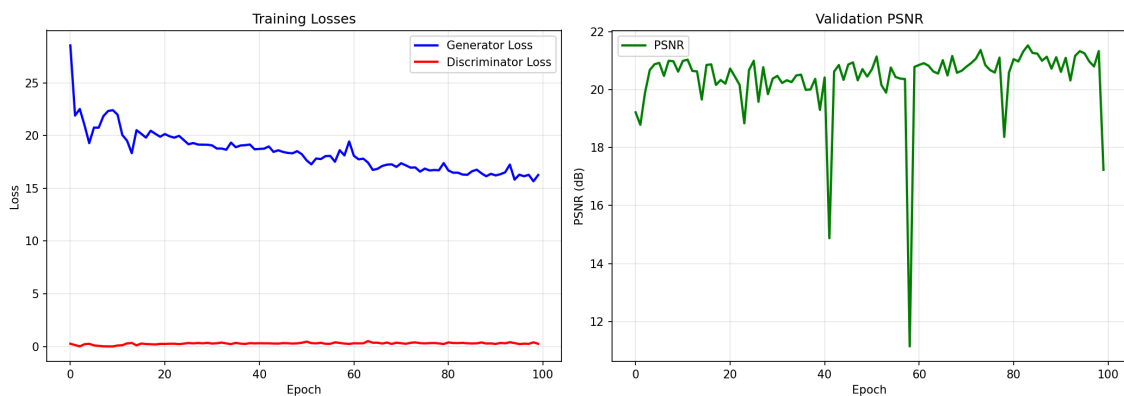
def forward(self, input_img, target_img):
    # 拼接有雾图和目标图（生成图或清晰图）
    combined = torch.cat([input_img, target_img], dim=1)
    return self.model(combined)

```

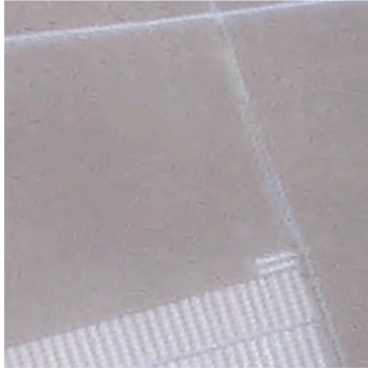
提交材料:

1. 项目源码
2. 可视化去雾效果
3. 一份文档，其中包含
 - 模型训练的**生成器损失函数(G_loss)**,**生成器损失函数(D_loss)**,**PNSR**的绘图
 - **(如有完成提高部分)**PNSR及**推理速度**分析
 - **(如有修改模型结构)**模型改进方法及实验分析

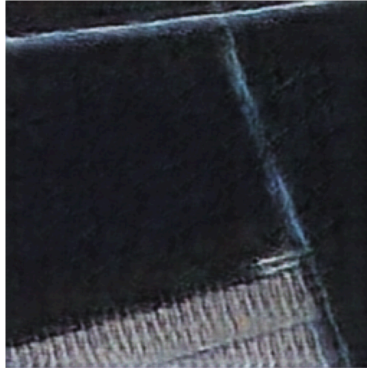
推荐的模型结构的相关数据



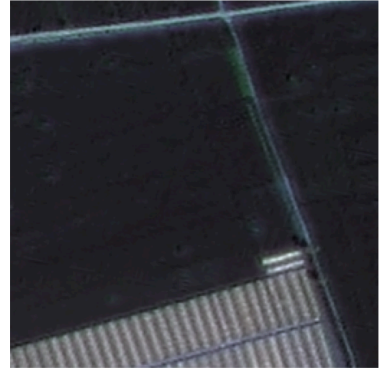
Sample 1: Hazy Input



Sample 1: Dehazed Output



Sample 1: Clear Target



Sample 2: Hazy Input



Sample 2: Dehazed Output



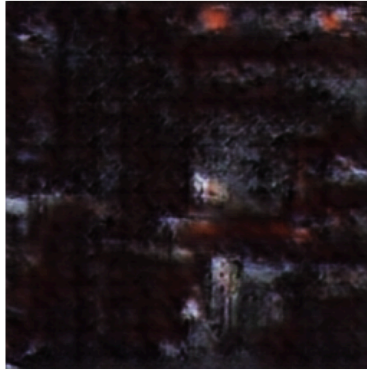
Sample 2: Clear Target



Sample 3: Hazy Input



Sample 3: Dehazed Output



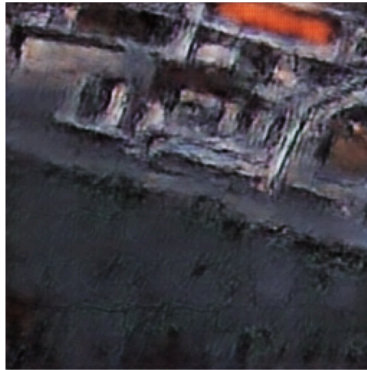
Sample 3: Clear Target



Sample 4: Hazy Input



Sample 4: Dehazed Output



Sample 4: Clear Target



推荐数据集: 去雾图像数据集 <https://www.modelscope.cn/datasets/LunaAmandatio/dahazing>

注:你可以选择其余的去雾数据集进行训练

提交方式:

请将除代码外文件(如视频,图片,报告)打包发送到 3813579841@qq.com,代码请在github建立一个Public的仓库,并按**班别-姓名-学号-仓库链接**的格式发送到题目仓库(https://github.com/LunaAmandatio/ComSen2026_Algorithm_Recruitment/tree/master) 的Issues下。

参考资料

1. 良好的版本控制习惯可以有效避免“改了一下午代码后它跑不起来了,但是我又改不回去了”之类的极端情况。推荐使用 [Git](#)
2. 奶龙也能看懂的pytorch入门教程《[PyTorch深度学习实践》完结合集](#)哔哩哔哩bilibili
3. 正确的使用项目结构可以提高代码可读性与更好的排查问题【[Python进阶系列](#)】第10篇: [Python项目的结构设计与目录规范——从脚本到模块,从混乱到整洁](#) - 知乎
4. 不清楚神经网络结构,这里有各种网络简介 [人工智能深度学习100种网络模型,精心整理,全网最全,PyTorch框架逐一搭建](#) - 知乎
5. YOLO原理解析【[一次看懂](#)】YOLO物体检测是如何工作的 | 算法解析 哔哩哔哩bilibili
6. 理解YOLO的 plots 参数生成的各图表含义YOLO模型训练报告解读 哔哩哔哩bilibili
7. 神经网络并不是只能用python构建, [Libtorch开发环境搭建](#) - 知乎
8. 常见数据集网站: **魔塔社区**, **Hugging Face**, **Roboflow**